Game-Theoretic Conflict Resolution

Exactly as implemented in `conflict_game.py` (Dubins_TSP_New_Implementation_claude). Numbers are the defaults in config.py; equation and step numbers refer to Dubins_TSP.pdf.

1. Conflict Detection

Positions are sampled every $\Delta t = 0.5$ s; between samples the sinks move in straight lines. For sinks i, j with relative position $r_{ij} = p_i - p_j$, the closest distance in interval n is

$$d_{ij}[n] = \min_{0 \le \tau \le 1} \; \| \, r_{ij}[n] + \tau \left(r_{ij}[n+1] - r_{ij}[n] \right) \| \,$$

The interval is unsafe ($u_{ij}[n] = 1$) if $d_{ij}[n] < d_s$ (= 3 m), unless both sinks are within 5 m of the base at that closest moment. It is checked only while both sinks are on their mission, $t \le \min(T_i, T_j)$ (eq. 20). The conflict exposure is the unsafe time from the decision frame s on:

$$E_{ij} = \Delta t \sum_{n \,\ge\, s} u_{ij}[n]$$

2. When a Game Starts

Let f be the earliest unsafe interval. The players N are the sinks that are unsafe in interval f. They decide $L = 5$ frames (2.5 s) earlier, at $s = \max(1, f - L)$; all other sinks keep their trajectories. After the game the search continues from $f + 1$, so conflicts are solved one game at a time, in time order (at most 500 games; Section 0.11, Steps 8–10).

3. Players and Strategies

Each player has a strategy set $A_k \subseteq \{\text{GO, WAIT, DETOUR}\}$, built once before the game; GO is always available.

- **GO**: continue along the current trajectory.
- **WAIT**: stop at the position of frame s for w_k frames, then continue. w_k is the shortest stop with the lowest risk against all other sinks, and $w_k \Delta t \le \min(20 \text{ s}, B - T_k)$.
- **DETOUR**: side-step $h = d_s + 0.5$ m = 3.5 m to the right (left only if right is infeasible) while the conflict lasts, then rejoin the own path. Offered only if the sink is moving, the ramps are at least $\pi\sqrt{(hR/2)}$ long, the turn stays within $1.15/R$, and the path keeps r_{safe} from all obstacles.
- **Budget rule** (Section 0.10, step 6): a strategy with $T_k + \Delta T_k > B$ is removed.

4. Cost Function

Each player minimises its own cost:

$$J_k = \alpha \, \rho_k + \beta \, \Delta T_k + \gamma \, \frac{\Delta D_k}{v} + \delta \, W_k$$

$$\rho_k = \sum_{j \,\in\, N,\; j \,\neq\, k} E_{kj} + B_k$$

ρ_k: collision-risk exposure with the other players and with the non-players (B_k). ΔT_k, ΔD_k, W_k: extra time, extra distance and waiting time of the chosen strategy (GO: all 0; WAIT: $\Delta T_k = W_k = w_k \Delta t$; DETOUR: ΔD_k = extra path length, $\Delta T_k \ge 0$). All terms are in seconds; $\beta = \gamma = \delta = 1$. With $c_k = \beta \Delta T_k + \gamma \Delta D_k / v + \delta W_k$, the collision weight is set for each game as

$$\alpha = \frac{1}{\Delta t} \left(1 + \sum_{k \,\in\, N} \left[\max_{A_k} c_k - \min_{A_k} c_k \right] \right)$$

so one interval of risk (Δt) costs more than any possible saving: no player accepts more risk to save time, distance or waiting.

5. Best Response and Nash Equilibrium

$$s_k^* = \arg\min_{s_k \,\in\, A_k} \; J_k(s_k, s_{-k})$$

Everyone starts on GO. In each round every player computes its best response and its gain; only the player with the largest gain switches (ties: shorter tour, then lower index). The process stops when no player can lower its cost by more than $\varepsilon = 10^{-9}$ by changing alone: a **pure Nash equilibrium**.

6. Potential Function

$$\Phi = \alpha \left(\sum_{i<j} E_{ij} + \sum_k B_k \right) + \sum_k c_k \qquad\qquad \Delta\Phi = \Delta J_k \;\; (\text{only player } k \text{ changes})$$

This exact potential property holds because (i) $E_{ij} = E_{ji}$: the danger is charged equally to both sinks; (ii) each strategy changes only its own sink's trajectory and is fixed before the game; (iii) non-players do not change during the game; (iv) α, β, γ, δ stay constant within a game. Every switch therefore lowers Φ, no strategy combination is visited twice, and best response stops at a pure Nash equilibrium after at most $\Pi_k |A_k| - 1$ moves.

Important limitation: the equilibrium is stable, but not necessarily conflict-free, and not necessarily the combination with the lowest Φ. Remaining conflicts are reported by the code, not hidden.